Handling Errors in Python

try / except — write code that survives the unexpected, instead of crashing on it.

Why Exception Handling?

The problem

- Bad input, missing files, network hiccups — things go wrong at runtime.
- An unhandled exception stops the program immediately, wherever it happens.
- We want to anticipate failure and recover gracefully instead of crashing.

```
age = int(input("Enter age: "))
print(f"You are {age} years old")
```

```
ERROR
Enter age: twenty
Traceback (most recent call last):
  File "app.py", line 1, in <module>
    age = int(input("Enter age: "))
ValueError: invalid literal for int() with base 10: 'twenty'
```

- **Note:** an uncaught exception halts the whole program instantly — nothing after it runs.

Basic try / except Syntax

How it's written

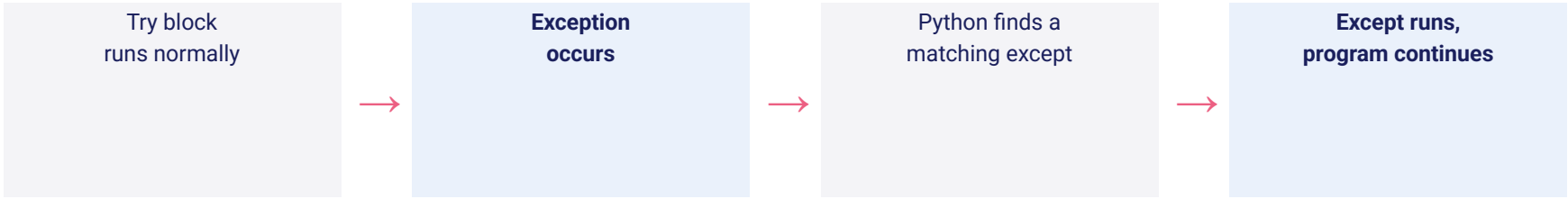
- Code that might fail goes inside the try block.
- If an exception occurs, control jumps to the matching except block.
- The program keeps running afterward instead of crashing.

```
try:
    age = int(input("Enter age: "))
    print(f"You are {age} years old")
except ValueError:
    print("That's not a valid number")
```

OUT

```
Enter age: twenty
That's not a valid number
```

How It Flows



- If no exception occurs, every except block is skipped entirely.
- Only the first matching except runs — Python stops looking after that.
- Code after the whole try/except block always continues, either way.

Catching Specific Exceptions

Name the exception

- Catch only the exception type you expect — never a bare `except:`.
- Different problems get different except blocks, each with its own fix.
- Unexpected errors still crash loudly instead of being hidden by accident.

```python
try:
    with open("config.json") as f:
        data = f.read()
except FileNotFoundError:
    print("Config file not found - using defaults")
except PermissionError:
    print("No permission to read config file")
```

Caution: avoid bare except: — it silently catches everything, including typos, making bugs hard to find.

Catching Multiple Exceptions

One block, several types

- Group related exception types in a tuple: except (KeyError, ValueError):
- Use this when the same fix applies to more than one failure mode.
- Keep unrelated exceptions in separate except blocks with their own handling.

```
try:
    value = int(data["amount"])
except (KeyError, ValueError):
    print("Amount is missing or not a valid number")
```

The Exception Object

Reading the error

- except ValueError as e: captures the exception into a variable.
- str(e) gives the error message Python generated for that exception.
- type(e).__name__ tells you the exact exception class that was raised.

```
try:
    price = float(user_input)
except ValueError as e:
    print(f"Could not convert: {e}")
    print(f"Error type: {type(e).__name__}")
```

OUT

```
Could not convert: could not convert string to float: 'abc'
Error type: ValueError
```

Exercise

Digits & Length

1 Numbers only

- Ask the user to input a phone number
- If it is not a number, print an error and ask again
- Stop only when the input converts with int()

```
is_not_number = True
while is_not_number:
    raw = input("Enter phone number: ")
    try:
        # TODO: int(raw) -> print OK, stop loop
    except ValueError:
        # TODO: print "Digits only, please"
```

Expected output:

abc → Digits only, please. (ask again)
012345678 → OK

2 Fixed length input

- Ask the user to input a phone number (once, no loop)
- If it is a number, check len() is 9 or 10
- Print OK, "Need 9-10 digits", or the error

```
raw = input("Enter phone number: ")
try:
    # TODO: convert raw with int()
    # TODO: if len is 9 or 10 -> print "OK"
    #       else print "Need 9-10 digits"
except ValueError:
    # TODO: print "Digits only, please"
```

Expected output:

0123 → Need 9-10 digits.
0123456789 → OK

Carrier & Bulk Check

3 Phone carrier prefix

- Use the carrier dict: each name holds a list of prefixes
- Ask the user to input a phone number
- Search for number[0:3] — if no name has it, catch KeyError

```
carrier = {"smart": ["010","070","016"],
           "cellcard": ["011","012","092"]}
try:
    # TODO: input a phone number
    # TODO: print the name that owns number[0:3]
except KeyError:
    # TODO: print "Unknown carrier"
```

Expected output:

0921234567 → cellcard
0161234567 → smart
0991234567 → Unknown carrier

4 Check many numbers

- Ask the user to input how many numbers to check
- Then ask the user to input that many phone numbers
- If the count is not a number, print an error

```
try:
    # TODO: input "How many numbers? "
    # TODO: convert the answer with int()
    # TODO: repeat that many times:
    #       input "Phone: " and print it
except ValueError:
    # TODO: print "Please type a number"
```

Expected output:

How many numbers? two → Please type a number
How many numbers? 2 → asks Phone: twice